ТЕХНІЧНЕ ЗАВДАННЯ

1. Загальні відомості

Назва програмного продукту: Formula 1 Race Strategy Simulator
Тип програмного забезпечення: десктопний застосунок для персонального комп'ютера
Платформа: Windows
Мова програмування: C++
Графічна бібліотека / оболонка: Qt
Тип інтерфейсу: графічний інтерфейс користувача (GUI)
Стиль програми: інтерактивний симулятор гоночної стратегії

2. Призначення програми

Програма призначена для моделювання гонки Формули-1 у спрощеному вигляді з можливістю візуального спостереження за рухом болідів по трасі та прийняття користувачем стратегічних рішень під час гонки.

Застосунок дозволяє:

- створювати та запускати симуляцію гонки;
- обирати трасу, болід, гонщика або команду;
- спостерігати за рухом болідів по трасі;
- контролювати стан шин, температуру та рівень пального;
- приймати рішення щодо темпу руху та піт-стопів;
- переглядати поточні позиції учасників гонки;
- отримувати фінальний результат симуляції.

3. Мета розробки

Метою розробки є створення десктопного застосунку, який демонструє принципи об'єктно-орієнтованого програмування на прикладі симуляції гонки Формули-1.

Програма повинна забезпечувати:

- наочне графічне відображення гонки;
- інтерактивну взаємодію користувача із симуляцією;
- використання класів та об'єктів для представлення елементів системи;
- розділення логіки симуляції та графічного відображення;
- можливість моделювання впливу шин, двигуна, гонщика та стратегії на результат гонки.

4. Функціональні вимоги

4.1 Керування симуляцією гонки

Програма повинна забезпечувати можливість:

- створення нової гонки;
- запуску симуляції;
- паузи симуляції;
- покрокового або безперервного проходження гонки;
- завершення симуляції.

4.2 Налаштування параметрів гонки

Користувач повинен мати можливість:

- обирати трасу;
- обирати гонщика або команду;
- обирати стартовий тип шин;
- задавати початковий режим їзди.

Можливі режими їзди:

- **Push** — агресивний темп;
- **Normal** — стандартний темп;
- **Conserve** — економний темп.

4.3 Симуляція руху болідів

Програма повинна моделювати рух болідів по трасі.

Для кожного боліда повинні враховуватися:

- поточна позиція на трасі;
- швидкість проходження кола;
- поточний номер кола;
- загальний час гонки;
- позиція у загальному заліку.

Візуалізація повинна відображати:

- трасу;
- боліди, що рухаються по ній;
- зміну позицій учасників;
- події піт-стопів.

4.4 Моделювання стану шин

Для кожного боліда повинна зберігатися та оновлюватися інформація про шини:

- тип шин (Soft / Medium / Hard);
- рівень зносу;
- температура шин;
- вплив шин на темп боліда.

Програма повинна враховувати:

- швидший знос м’яких шин;
- повільніший знос жорстких шин;
- вплив агресивного стилю їзди на знос;
- вплив температури шин на швидкість.

4.5 Піт-стопи

Користувач повинен мати можливість виконати піт-стоп під час гонки.

При піт-стопі програма повинна:

- тимчасово зменшувати темп боліда;
- додавати штраф часу;
- змінювати комплект шин;
- оновлювати характеристики боліда після заїзду в бокси.

4.6 Відображення поточної інформації

Під час симуляції програма повинна відображати:

- номер кола;
- позицію користувача;
- тип шин;
- рівень зносу шин;
- температуру шин;
- поточний режим їзди;
- відставання або перевагу відносно суперників;
- таблицю позицій усіх болідів.

4.7 Визначення результатів гонки

Після завершення симуляції програма повинна відображати:

- фінальну таблицю результатів;
- місце кожного учасника;
- загальний час гонки;
- кількість піт-стопів;
- інформацію про використані типи шин;
- переможця гонки.

5. Вимоги до інтерфейсу користувача

Програма повинна мати графічний інтерфейс користувача.

Додано примітку [1]: подумайте, чи встигнете зробити графічний інтерфейс, адже часу не так багато, а функціональності... дуже... багато...

Основні елементи інтерфейсу

Головне вікно програми

Головне вікно повинно містити:

- назву програми;
- кнопку створення нової гонки;
- кнопку запуску симуляції;
- кнопку виходу;
- панель налаштування параметрів гонки.

Вікно або панель налаштування гонки

Повинно містити:

- вибір траси;
- вибір гонщика або команди;
- вибір стартового комплекту шин;
- вибір режиму їзди.

Вікно симуляції

Повинно містити:

- графічне зображення траси;
- боліди, які рухаються по трасі;
- інформаційну панель стану користувача;
- таблицю поточних позицій;
- кнопки керування.

Кнопки керування

Програма повинна містити кнопки або команди:

- **Start Race**
- **Pause**
- **Continue**
- **Push**
- **Normal**
- **Conserve**
- **Pit Stop**
- **Exit**

6. Архітектура програми

Програма повинна бути реалізована з використанням принципів об’єктно-орієнтованого програмування.

У програмі повинні використовуватися класи.

Основні класи

Клас Driver

Призначення: зберігання інформації про гонщика.

Поля класу:

- ім’я гонщика;
- рівень майстерності;
- рівень стабільності;
- рівень керування зносом шин.

Методи:

- отримання інформації про гонщика;
- обчислення бонусу до темпу;
- обчислення впливу на знос шин.

Клас Engine

Призначення: зберігання характеристик двигуна боліда.

Поля класу:

- назва двигуна;
- потужність;
- надійність.

Методи:

- отримання впливу потужності на швидкість;
- отримання інформації про двигун.

Клас Tyre

Призначення: зберігання та оновлення стану шин.

Поля класу:

- тип шин;
- рівень зносу;
- температура;
- коефіцієнт зчеплення.

Методи:

- збільшення зносу;
- зміна температури;
- обчислення впливу шин на темп;
- заміна комплекту шин.

Клас F1Car

Призначення: представлення боліда.

Поля класу:

- модель боліда;
- двигун;
- поточний комплект шин;
- вага;
- поточна швидкість;
- позиція на трасі.

Методи:

- обчислення часу кола;
- оновлення параметрів боліда;
- виконання піт-стопу;
- отримання поточної позиції.

Клас Track

Призначення: представлення траси.

Поля класу:

- назва траси;
- довжина траси;
- кількість кіл;
- коефіцієнт зносу шин;
- базовий час кола.

Методи:

- отримання параметрів траси;
- вплив траси на темп та знос шин.

Клас Strategy

Призначення: представлення поточного стилю гонки.

Поля класу:

- режим їзди;
- момент піт-стопу;
- поточна тактика.

Методи:

- зміна режиму їзди;
- визначення необхідності піт-стопу;
- отримання модифікатора темпу.

Клас Race

Призначення: керування процесом симуляції гонки.

Поля класу:

- траса;
- список учасників;
- поточне коло;
- стан гонки.

Методи:

- запуск гонки;
- оновлення стану гонки;
- розрахунок результатів;
- визначення позицій;
- завершення гонки.

Клас Renderer

Призначення: графічне відображення елементів симуляції.

Методи:

- відображення траси;
- відображення болідів;
- відображення таблиці результатів;
- оновлення інтерфейсу.

Клас GameController

Призначення: зв’язок між логікою програми та користувацьким інтерфейсом.

Методи:

- обробка дій користувача;
- запуск сценаріїв програми;
- передача команд до симуляції;
- керування станом застосунку.

7. Вимоги до реалізації

Під час реалізації програми повинні використовуватися:

- класи та об’єкти;
- конструктори;
- деструктори;
- інкапсуляція;
- методи класів;
- контейнери для зберігання даних (vector);
- композиція об’єктів;
- наслідування або поліморфізм (за потреби для стратегій);
- взаємодія графічного інтерфейсу з логікою програми;
- багатофайлова структура проєкту.

На початковому етапі програмна логіка може бути реалізована та протестована у консольному режимі з подальшою інтеграцією у графічний інтерфейс.

8. Розподіл обов’язків у команді

Розробник програмної логіки

Відповідає за реалізацію внутрішньої структури програми та основного функціоналу симулятора.

Основні задачі:

- проектування структури класів;
- реалізація основних класів предметної області;
- створення логіки симуляції гонки;
- реалізація функцій обчислення часу кола;
- реалізація логіки шин, двигуна, темпу та піт-стопів;
- реалізація алгоритмів оновлення стану гонки;
- реалізація класу GameController;
- підготовка консольної тестової версії функціоналу;
- тестування правильності роботи логіки програми.

Основні компоненти, за які відповідає:

- Driver
- Engine
- Tyre
- F1Car
- Track
- Strategy
- Race
- GameController

Розробник графічного інтерфейсу та репозиторію

Відповідає за створення графічної частини застосунку, інтеграцію логіки з інтерфейсом і супровід репозиторію проєкту.

Основні задачі:

- розробка графічного інтерфейсу користувача;
- створення вікон, кнопок, панелей і елементів керування;

- реалізація візуального відображення траси та болідів;
- інтеграція графічного інтерфейсу з програмною логікою;
- забезпечення взаємодії користувача з функціями симулятора;
- організація репозиторію проєкту;
- ведення гілок, оновлення коду та контроль структури GitHub-репозиторію;
- інтеграція змін від обох розробників.

Основні компоненти, за які відповідає:

- MainWindow
- Renderer / графічні елементи інтерфейсу
- елементи керування симуляцією
- відображення таблиць, панелей стану та трас
- GitHub-репозиторій та інтеграція змін

9. Терміни розробки

Термін виконання проєкту: 1 місяць

Етапи виконання:

1. Аналіз предметної області та проектування структури програми
2. Розробка основних класів
3. Реалізація логіки симуляції
4. Розробка графічного інтерфейсу
5. Інтеграція логіки та графічного відображення
6. Тестування програми
7. Підготовка демонстраційної версії

10. Очікуваний результат

Результатом роботи повинен бути десктопний застосунок, який:

- запускається на персональному комп’ютері;
- має графічний інтерфейс користувача;
- моделює гонку Формули-1;
- відображає рух болідів по трасі;
- дозволяє користувачу приймати стратегічні рішення;
- демонструє вплив шин, темпу та піт-стопів на результат;
- формує фінальну таблицю результатів гонки

11. Перспективи розвитку проєкту

За потреби в майбутньому програму можна розширити шляхом додавання:

- декількох трас;
- декількох типів стратегій;
- погодних умов;
- режиму кваліфікації;
- збереження результатів гонки;
- статистики виступів користувача;
- більш деталізованої візуалізації.

12. Етапи реалізації та рівні вимог

Додано примітку [2]: бомба-пушка! особливо в реальних ТЗ це треба робити, це так звані "milestones"
супер!

З огляду на обсяг функціоналу, реалізація проєкту поділяється на кілька рівнів складності. Це дозволяє поетапно розробляти програму та забезпечити отримання працездатного результату навіть у разі обмеженого часу.

12.1 Мінімальний рівень реалізації

Мінімальний рівень повинен забезпечити базову працездатність симулятора.

До мінімального рівня входять:

- реалізація основних класів програми;
- створення базової логіки симуляції гонки;
- моделювання проходження гонки по колах;
- обчислення часу кола для кожного боліда;
- врахування характеристик гонщика, двигуна та шин;
- знос шин;
- витрата пального;
- базова логіка піт-стопів;
- визначення позицій учасників;
- консольне або базове текстове відображення результатів симуляції.

12.2 Середній рівень реалізації

Середній рівень передбачає розширення базового функціоналу та покращення зручності сприйняття симуляції.

До середнього рівня входять:

- покращене відображення поточного стану гонки;
- виведення таблиці позицій;
- відображення відставання від лідера;
- відображення режиму їзди;
- відображення температури та зносу шин;
- відображення подій піт-стопів;
- інтерактивне керування симуляцією;
- графічне або напівграфічне відображення гонки.

12.3 Підвищений рівень реалізації

Підвищений рівень охоплює додаткові можливості, які можуть бути реалізовані за наявності часу.

До підвищеного рівня можуть входити:

- декілька трас;
- декілька типів стратегій;
- зміна режиму їзди під час гонки;
- розширена візуалізація траси;
- відображення руху болідів у реальному часі;
- статистика гонки;
- режим кваліфікації;
- погодні умови;
- збереження результатів.

12.4 Принцип поетапної реалізації

Розробка проєкту здійснюється поетапно:

1. реалізація мінімального працездатного функціоналу;
2. розширення симуляції до середнього рівня;
3. додавання додаткових можливостей за наявності часу.

Такий підхід дозволяє забезпечити завершеність проєкту, контроль обсягу роботи та поступове ускладнення функціоналу.